

КТ-2.1 — Архитектура системы

Technical Design Document

Кадочников Лев Петрович (гр. МФТИ-1-2024)

Команда: «Черепашки Ninja»

2026-06-13

1. Общая схема архитектуры (System Context / Container Diagram)

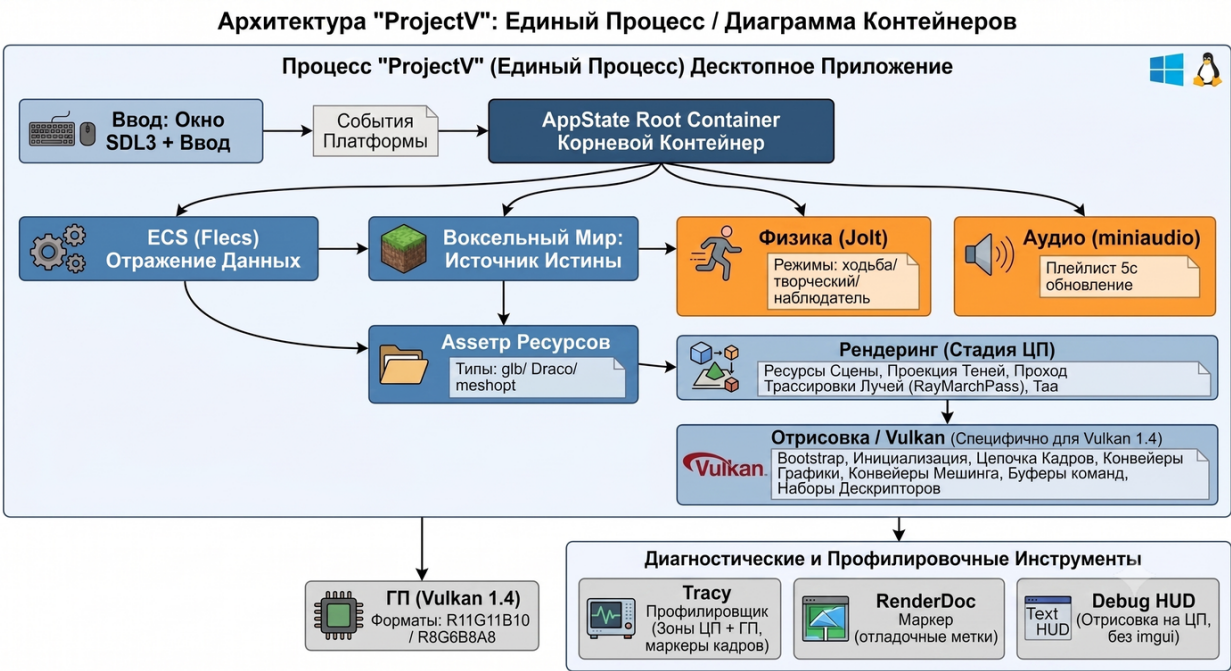
ProjectV — десктопное приложение реального времени (нативный бинарник под Windows и Linux). Архитектура однопроцессная, не клиент-серверная: вся логика, рендер и физика выполняются в одном процессе на стороне пользователя.

Подсистемы первого уровня:

Подсистема	Зона ответственности	Зависимости
Платформа	Окно (SDL3), ввод, жизненный цикл	SDL3
Воксельный мир	Источник истины для всех воксельных данных	—
Физика	Walk-персонаж, столкновения со статической геометрией	Jolt
Аудио	Фоновая музыка, плейлист, горячая замена треков	miniaudio
ECS	Зеркало состояния мира для запросов и симуляций	flecs
Реестр ассетов	glTF / Draco / meshopt-ассеты, горячая перезагрузка	fastgltf, draco, meshopt
Рендерер	Мешинг, освещение, тени, TAA, ray march, debug-оверлей	Vulkan 1.4
GPU	Все вычислительные и графические ресурсы	—

Поток данных (упрощённо): SDL3 → события платформы → InputState → AppState → VoxelWorld (источник истины) → Физика (Jolt) + ECS (зеркало) + Реестр ассетов → Рендерер (подготовка на CPU) → Vulkan → GPU.

Графическая схема архитектуры (System Context / Container Diagram):



2. Стек технологий с обоснованием

Языки и сборка:

Технология	Версия	Назначение
C++26	ISO/IEC 14882:2026	Основной язык реализации
C23	ISO/IEC 9899:2024	Подмодули с C-API (Jolt, draco, meshopt)

Технология	Версия	Назначение
CMake	3.30+	Система сборки
Ninja	1.13+	Бэкенд сборки (быстрые инкрементальные сборки)
Clang 22	22.1.6	Компилятор (мультиплатформенный, C++26-фичи)

Платформа и графика:

Технология	Версия	Назначение
SDL3	3.4.10	Платформенный слой (окно, ввод)
Vulkan 1.4	1.4.350	Графический API
volk	pinned	Vulkan-загрузчик (runtime-загрузка)
VMA	pinned b3cbbb43	Менеджер памяти GPU (chronos-group)
GLSL → SPIR-V	4.5 / 1.4	Шейдеры (автокомпиляция через glslc)
RenderDoc	1.30+	GPU-отладчик

Сторонние библиотеки:

Технология	Версия	Назначение
flecs	2.2.0	ECS (header-only, MIT)
JoltPhysics	pinned	Физика (детерминированная)
Tracy	0.11+	Профилирование (CPU + GPU-зоны)
fmt	pinned	Типобезопасное форматирование
glm	pinned	Математика (header-only)
fastgltf	pinned	glTF-загрузчик (header-only, simdjson backend)
draco	pinned	Сжатие мешей (Google decoder-only)
meshoptimizer	pinned	Оптимизация мешей (vertex cache, перерисовка)
miniaudio	pinned	Аудио (header-only MIT, single-file)
nlohmann/json	3.11.3 (FetchContent)	JSON-парсинг (header-only MIT)
Dear ImGui	opt-in (OFF по умолчанию)	ImGui UI

Итого: 22 подмодуля + 1 FetchContent (nlohmann/json) + системные библиотеки (zlib, png, jpeg для текстур — opt-in).

3. Структуры данных

Структура	Поля (типы)
VoxelWorld	voxels: vector<uint8_t>, chunks: vector<...>
VoxelChunk	min/maxExclusive: Int3, rebuildQueued: bool, nonAirVoxelCount: Int
VoxelMaterial (enum)	Air=0, Glass=1, Fluid=2, FloorWhite=3, FloorGray=4 (uint8)
VoxelMaterialVisual (struct)	baseColor: vec4, surface: {A0, roughness, metallic, reflectance}
AppState (root)	см. §4
WorldState	voxelWorld: unique_ptr<VoxelWorld>, requestedScenePreset: VoxelScene
SceneResources	chunkDescriptors: vector<...>, sceneChunkVoxelData: ...
InteractionState	selection: RaycastHit, placementVoxel: Int3, hitNormal: vec3, hitDistance: float
PackedSceneChunkDescriptor	64 B std430 — см. детали ниже
PackedSceneVoxelFace	16 B — см. детали ниже
VoxelSceneLighting	9 × array<float, 4> — см. детали ниже

Детализация критичных структур:

- **PackedSceneChunkDescriptor** (64 B, std430, см. src/core/Types.hpp:110-118):
 - chunkOrigin: array<int32, 4> (offset 0) — начало чанка в мировых координатах;
 - chunkExtentAndNonAir: array<uint32, 4> (offset 16) — размеры + счётчик non-air;

- voxelDataInfo: array<uint32, 4> (offset 32) — индексы в sceneChunkVoxelData;
- drawRanges: array<uint32, 4> (offset 48) — диапазоны draw call'ов.
- **PackedSceneVoxelFace** (16 B, см. src/core/Types.hpp:96-108):
 - voxelCoord, axisMask, materialIndex, lightingData, packedExtents: uint32_t — координаты вокселя, маска осей, материал, освещение, packed extents.
- **VoxelSceneLighting** (9 × vec4, контракт с GLSL SceneLightingBuffer):
 - skyColorAndFogDensity, horizonColorAndFogStart, groundColorAndFogMax, sunColorAndIntensity, sunDirectionAndWrap, postProcess, sunShadowParams, sunContactShadowParams, ambientOcclusionParams.

Связи (FK):

- VoxelWorld → VoxelChunk (1:N) — каждый чанк принадлежит одному миру.
- VoxelChunk ↔ VoxelWorld (1:1) — границы (min/max) задают подмножество мировых координат.
- VoxelMaterialVisual → VoxelMaterial (N:1) — несколько визуалов могут ссылаться на один материал.
- WorldState → VoxelWorld (1:1) — unique_ptr.
- SceneResources → VoxelChunk (1:1) — chunkDescriptors[i] соответствует chunks[i].
- PackedSceneChunkDescriptor → VoxelChunk.min (1:1) — chunkOrigin задаёт начало чанка.
- PackedSceneVoxelFace → VoxelWorld.voxels (N:1) — грань ссылается на конкретный воксель.
- VoxelSceneLighting — общий контракт с шейдерами (GLSL SceneLightingBuffer).

4. AppState — корневой контейнер

AppState — единый корневой объект, через который проходят все данные между подсистемами.

Поле	Тип
platform	PlatformState
context	VulkanContextState
swapchain	SwapchainState
world	WorldState
render	RenderState
input	InputState
interaction	InteractionState
simulation	SimulationState
ecs	EcsStatePtr (unique_ptr<flecs::world>)
physics	PhysicsStatePtr (unique_ptr<PhysicsWorld>)
audio	AudioEnginePtr (unique_ptr<AudioEngine>)
benchmark	BenchmarkAutomationState
lookDevCapture	LookDevCaptureAutomationState
frame	FrameState

Краткое назначение полей:

- platform — окно SDL3 и обработчик изменения размера.
- context — VkInstance, Device, очередь, аллокатор.
- swapchain — VkSwapchainKHR, представления изображений, extent.
- world — requestedScenePreset, указатель на VoxelWorld.
- render — sceneResources, frameData, отладочные оверлеи.
- input — дельта мыши, снимок действий, replay.

- `interaction` — попадание raycast, размещаемый воксель, нормаль и дистанция попадания.
- `simulation` — аккумулятор, `timeScale`, `frameStep`.
- `ecs` — мир flecs (ECS-зеркало).
- `physics` — мир Jolt.
- `audio` — аудиодвижок с пользовательским удалителем.
- `benchmark` — `frameCount`, средние, минимум/максимум.
- `lookDevCapture` — прогрев, список видов для съёмки.
- `frame` — профиль CPU, тайминги рендера.

Связи между ключевыми сущностями:

Связь	Кард. FK	Назначение
VoxelWorld → VoxelChunk	1:N editVersion	версия мира (инкремент при правке)
VoxelChunk ↔ границы	1:1 min/max	границы внутри VoxelWorld
SceneResources → VoxelChunk	1:1 chunkDescriptors[i]	индекс чанка в chunks
PhysicsWorld ↔ VoxelWorld	1:1 (read-only)	физ-мир из solid-вокселей
Renderer → SceneResources	1:1 —	рендер читает стейтинг
CharacterVirtual (Jolt) → AppState	1:1 —	корневой контейнер

5. Описание ключевых программных модулей

ProjectV декомпозирован на 11 независимых модулей:

Модуль	Зона ответственности
Платформа	Окно (SDL3), обработка событий, InputState
Воксельный мир	Хранение, редактирование, грязные чанки, raycast
Физика	Walk-персонаж (CharacterVirtual), Jolt, синхронизация
Аудио	Плейлист, 5-секундное автообновление, горячая замена
ECS	Зеркало мира для запросов (MIT/RAII)
Реестр ассетов	Конвейер glTF / Draco / meshopt, горячая перезагрузка
Рендерер (подготовка на CPU)	Мешинг, освещение, данные кадра
Проекция теней	Тень по сцене, CSM, контактная тень
Ray March	Параллельный GPU-проход для объёмного рендера
ТАА	Temporal Anti-Aliasing, исправление гонки дескрипторов
Vulkan (низкий уровень)	Instance, Device, Swapchain, конвейеры, командные буферы

Ключевые файлы и типы по модулям (для быстрой навигации):

1. Платформа — `src/platform/PlatformEvents.cpp`, `InputState.hpp`
2. Воксельный мир — `src/voxel/VoxelWorld.cpp`, `VoxelWorld.hpp`
3. Физика — `src/physics/PhysicsWorld.cpp`, `PhysicsWorld.hpp`
4. Аудио — `src/audio/AudioEngine.cpp`, `AudioEngine.hpp`
5. ECS — `src/ecs/EcsState.*`
6. Реестр ассетов — `src/asset/AssetManifest.cpp`, `AssetManifest.hpp`
7. Рендерер (подготовка на CPU) — `src/render/SceneResources.cpp`, `SceneResources.hpp`
8. Проекция теней — `src/render/ShadowProj.*`
9. Ray March — `src/render/RayMarchPass.*`
10. ТАА — `src/render/Taa.*`
11. Vulkan (низкий уровень) — `src/render/vulkan/*`

6. Описание ключевых алгоритмов

1. **Игровой цикл** (AppMain). Физика с фиксированным шагом (60 Гц), рендер с переменным шагом, профилирование через Tracy.
2. **Greedy-мешинг** (voxel_mesh.comp). Объединение соседних solid-вокселей в четырёхугольные грани для снижения перерисовки и indirect-draw.
3. **Полномочия на walk** (Физика + Воксельный мир). CharacterVirtual синхронизирован с VoxelWorld.editVersion; только физика запрашивает столкновения, остальные подсистемы читают VoxelWorld read-only.
4. **Тень по сцене** (Проекция теней + voxel_mesh.comp). Каскадные карты теней (CSM) рассчитаны по sceneChunkVoxelData, а не по пирамиде видимости — оптимизация под плотные воксельные сцены.
5. **Клеточный автомат жидкости** (voxel_fluid.comp). Двусторонний обмен с VoxelWorld для материала Fluid=2: каждый шаг СА читает соседние ячейки, при необходимости обновляет значение.